

EXPERIMENT 1: Text Preprocessing using NLTK

AIM:

To perform text preprocessing on various corpora using NLTK library. The experiment demonstrates normalization, cleaning, tokenization, stopwords removal, stemming, and lemmatization on Brown, Reuters, 20 Newsgroups, and UDHR datasets.

PROCEDURE:

1. Install required libraries: nltk, scikit-learn, pandas.
2. Download NLTK corpora: brown, reuters, stopwords, punkt, wordnet, omw-1.4.
3. Import required modules: re, nltk, brown, reuters, stopwords, word_tokenize, PorterStemmer, WordNetLemmatizer, fetch_20newsgroups.
4. Initialize stop_words, stemmer, and lemmatizer.
5. Define preprocess_text(text) that applies:
 - Text Normalization (lowercase)
 - Text Cleaning (remove punctuation and numbers using regex)
 - Tokenization (word_tokenize)
 - Stopword Removal
 - Stemming (PorterStemmer)
 - Lemmatization (WordNetLemmatizer)
6. Apply preprocessing to sample text from each corpus (Brown, Reuters, 20Newsgroups, UDHR).
7. Print tokens, stemmed, and lemmatized outputs for each.

CODE:

```
pip install nltk scikit-learn pandas

import nltk
nltk.download('brown') nltk.download('reuters')
nltk.download('stopwords')
nltk.download('punkt') nltk.download('wordnet')
nltk.download('omw-1.4')

import re
import nltk
from nltk.corpus import brown, reuters, stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.datasets import fetch_20newsgroups
import pandas as pd

stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    # 1. Text Normalization (lowercase)
    text = text.lower()
    # 2. Text Cleaning (remove punctuation and numbers)
    text = re.sub(r'[^\w\s]', '', text)
    # 3. Tokenization
    tokens = word_tokenize(text)
    # 4. Stopword Removal
    tokens = [word for word in tokens if word not in stop_words]
    # 5. Stemming
    stemmed = [stemmer.stem(word) for word in tokens]
    # 6. Lemmatization
    lemmatized = [lemmatizer.lemmatize(word) for word in tokens]
    return {"tokens": tokens, "stemmed": stemmed, "lemmatized": lemmatized}

# Brown corpus
brown_text = " ".join(brown.words()[:1000])
brown_processed = preprocess_text(brown_text)
print("Tokens:", brown_processed["tokens"][:20])
print("Stemmed:", brown_processed["stemmed"][:20])
print("Lemmatized:", brown_processed["lemmatized"][:20])

# Reuters corpus
reuters_text = " ".join(reuters.words()[:1000])
reuters_processed = preprocess_text(reuters_text)
print("Tokens:", reuters_processed["tokens"][:20])
print("Stemmed:", reuters_processed["stemmed"][:20])
print("Lemmatized:", reuters_processed["lemmatized"][:20])

# 20 Newsgroups
newsgroups = fetch_20newsgroups(subset='train')
newsgroup_text = newsgroups.data[0]
newsgroup_processed =
```

```
preprocess_text(newsgroup_text) print("Tokens:",
newsgroup_processed["tokens"][:20]) print("Stemmed:",
newsgroup_processed["stemmed"][:20]) print("Lemmatized:",
newsgroup_processed["lemmatized"][:20])
```

```
# UDHR corpus
from nltk.corpus import udhr nltk.download('udhr')
ud_text = " ".join(udhr.words('English-Latin1')[:1000])
ud_processed = preprocess_text(ud_text) print("Tokens:",
ud_processed["tokens"][:20]) print("Stemmed:",
ud_processed["stemmed"][:20]) print("Lemmatized:",
ud_processed["lemmatized"][:20])
```

OUTPUT:

```
Tokens: ['fulton', 'county', 'grand', 'jury', 'said', 'friday', 'investigation',  
'atlanta', 'recent', 'primary', 'election', ...]  
Stemmed: ['fulton', 'counti', 'grand', 'juri', 'said', 'friday', 'investig', 'atlanta',  
'recent', 'primari', 'elect', ...]  
Lemmatized: ['fulton', 'county', 'grand', 'jury', 'said', 'friday', 'investigation',  
'atlanta', 'recent', 'primary', 'election', ...]
```

```
Tokens: ['asian', 'exporters', 'fear', 'damage', 'u', 'japan', 'rift', 'mounting',  
'trade', 'friction', ...]  
Stemmed: ['asian', 'export', 'fear', 'damag', 'u', 'japan', 'rift', 'mount', 'trade',  
'friction', ...]  
Lemmatized: ['asian', 'exporter', 'fear', 'damage', 'u', 'japan', 'rift', 'mounting',  
'trade', 'friction', ...]
```

```
Tokens: ['lerxstwamumdedu', 'wheres', 'thing', 'subject', 'car', 'nntppostinghost',  
'racwamumdedu', 'organization', ...]  
Stemmed: ['lerxstwamumdedu', 'where', 'thing', 'subject', 'car', 'nntppostinghost',  
'racwamumdedu', 'organ', ...]  
Lemmatized: ['lerxstwamumdedu', 'wheres', 'thing', 'subject', 'car', 'nntppostinghost',  
'racwamumdedu', 'organization', ...]
```

```
Tokens: ['universal', 'declaration', 'human', 'rights', 'preamble', 'whereas',  
'recognition', 'inherent', 'dignity', 'equal', ...]  
Stemmed: ['univers', 'declar', 'human', 'right', 'preambl', 'wherea', 'recognit',  
'inher', 'digniti', 'equal', ...]  
Lemmatized: ['universal', 'declaration', 'human', 'right', 'preamble', 'whereas',  
'recognition', 'inherent', 'dignity', 'equal', ...]
```

RESULT:

The text preprocessing pipeline was successfully implemented using NLTK. The experiment demonstrated that stemming reduces words to their root form (e.g., 'investig', 'counti') while lemmatization returns valid base forms (e.g., 'investigation', 'county'). The pipeline effectively removed stopwords, punctuation, and numbers from all four corpora.